

Levy-Distributed Noise and Loss-Based Adaptive Control with Markov Chain Monte Carlo Methods for Gaussian Splatting

1. Foreword

This document, and the repository it's in, was originally part of a class project for CS7641 Machine Learning at Georgia Tech. The code and documentation presented here is my individual work for the part of this project that was assigned to me, which was separate from the other parts.

This builds upon GSplat, a Gaussian Splatting package for Python, and so the original GSplat code is included in this repository. My work is included in a separate commit.

2. Previous Work

Gaussian splatting [7] is a machine learning technique which reconstructs scenes in 3D from a collection of static 2D images. Rather than using traditional mesh-based approaches, gaussian splatting uses point-based methods rendered with alpha blending.

In gaussian splatting, each individual point to render is represented by a 3D gaussian, which are optimized by gaussian splatting's machine learning algorithm. They can be thought of as colored orbs or ellipsoids in space, and have these parameters.

- Position in world coordinates
- Scaling in x, y, and z
- Rotation, as a quaternion
- Color and opacity, represented with RGBA

The rendering method for these gaussians was first described in [14], and was later used for the original gaussian splatting algorithm [7]. To summarize, we start by projecting the 3D covariance matrix Σ of this gaussian onto the 2D image plane. One piece needed for this is the world-view matrix W , which converts coordinates from world space to view/camera space. The second involves its projective transformation into the image's ray space – or more specifically, an approximation of that transformation's jacobian J . The result is the 2D covariance matrix Σ_{2D} .

$$\Sigma = R S S^T R^T$$

$$\Sigma_{2D} = J W \Sigma W^T J^T$$

The gaussians are then processed from furthest to nearest. The color that a gaussian contributes to the scene is determined using that gaussian's intrinsic color, opacity, and 2D covariance matrix, which is then composited to the scene using alpha blending. Alpha blending is an inherently differentiable process, and since our other operations are also differentiable, we can use machine learning algorithms to optimize each gaussian's parameters.

2.1 Adaptive Control Strategies

As training moves and recolors gaussians, they can sometimes get caught in places where there is no loss. These gaussians will be optimized to be completely transparent – the color is correct where it is, and so they should contribute no color at all. Unfortunately, this also means they have reached a local minima, and further optimization loops will not change them.

Every gaussian splatting implementation must handle this scenario somehow. We modify a Markov Chain Monte Carlo method detailed in [8], which we will call GS-MCMC. The original GS-MCMC handles these local minima in two ways.

First, it adds gaussian noise to the positions of transparent gaussians. The idea is that by perturbing their positions, they will eventually find themselves in a spot with loss, where the scene color does not match ground truth. The optimizer can then change its color and opacity to deal with this loss. The algorithm is based on Stochastic Gradient Langevin Dynamics.

Second, if this is not enough, it will periodically (e.g. every 100 epochs) find transparent gaussians and move them to the exact position of a random gaussian that is not transparent. It tries Brownian motion (as above) for a limited number of epochs, before restarting it from another position.

Our experiments make modifications to both these methods, and further modifies the regularizer that GS-MCMC uses. We use the Mip-NeRF 360 dataset [2] [4] for evaluation, which is one of the datasets used in the original gaussian splatting paper. Our code is built on top of GSplat [3], an open-source gaussian splatting package which comes with a base implementation of GS-MCMC, and extend it to use our own.

3. Modifications

We have experimented with changing three parts of the original GS-MCMC algorithm.

1. **Levy-distributed Langevin Dynamics** — The adaptive control strategy we modified uses a Stochastic Gradient Langevin Dynamics method that was introduced in [6] and [11]. It is usually implemented with Gaussian noise, and in general terms, this noise is added to gaussian positions in each gradient descent step.

$$\theta(t+1) = \theta(t) - \lambda_{lr} \left(\nabla E + \left(\lambda_{noise} \cdot \sigma(1 - \alpha) \cdot \epsilon \right) \right)$$

where α is opacity, and σ is the sigmoid function

It is important to note this is only applied to transparent gaussians – notice the sigmoid function uses opacity as input. While solid, opaque gaussians contribute color to the scene, a number of gaussians may be optimized to be transparent, and thus have no effect in the scene. These gaussians tend to be stationary, and are very unlikely to ever become opaque. The noise term ϵ is intended to move these gaussians to positions that aren't yet correct, trusting the gradient steps to optimize them out of transparency.

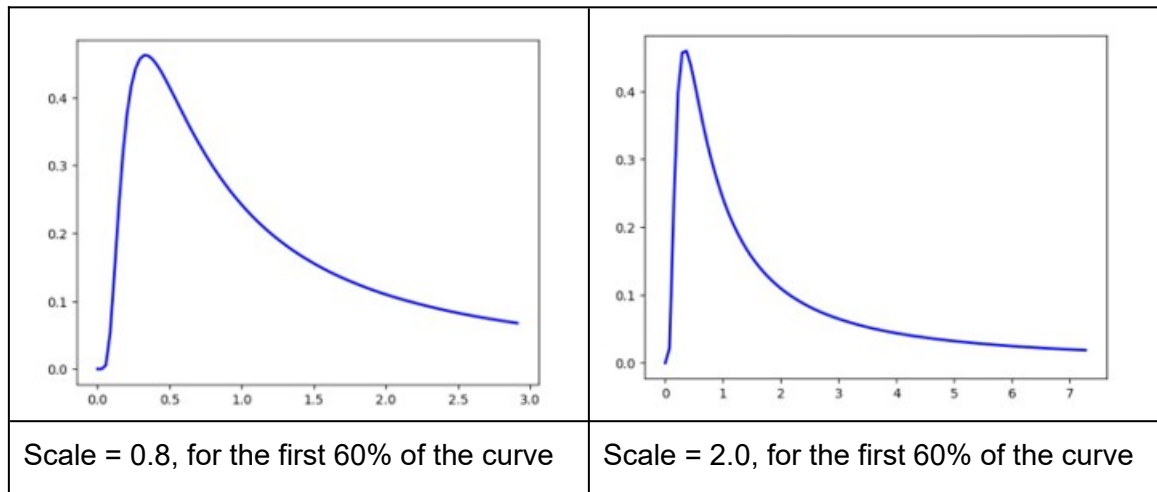
While this noise term ϵ is usually implemented (including by GSplat) as Gaussian noise with $N(\mu=0, \sigma^2=1)$, recent research suggests this is not the most optimal way to implement SGLD [5]. In our case, this research makes some intuitive sense. If a

gaussian is transparent, it is because its immediate vicinity has no loss, so it has been optimized to contribute no color where it is.

Moving a gaussian in the scene with this noise is likely to reposition it within this immediate vicinity, which is likely in an area with low loss. Alternatives have been presented in [5], [9], and [12] that make use of Levy alpha-stable distributions, which can be tuned to produce movements further away. We theorize this would increase the chance a transparent gaussian winds up in a space that happens to need more help (i.e. the position has higher error than its previous position).

We have opted to use the standard Levy distribution for our noise function. Since it only produces positive numbers, we also randomly generated a sign to each number. The Levy distribution is defined with a few parameters, which become hyperparameters.

- Location
 - The Levy distribution produces positive numbers starting at 0. This parameter can shift the curve horizontally.
- Scale
 - Horizontal scaling of the distribution curve.
- Percentage point function range
 - We use a percentage point function to create a noise value. In general terms, it accepts an input from (0.0, 1.0), and returns the value at that point of the distribution curve. For a normal distribution, $\text{ppf}(0.5)$ would return the mean.

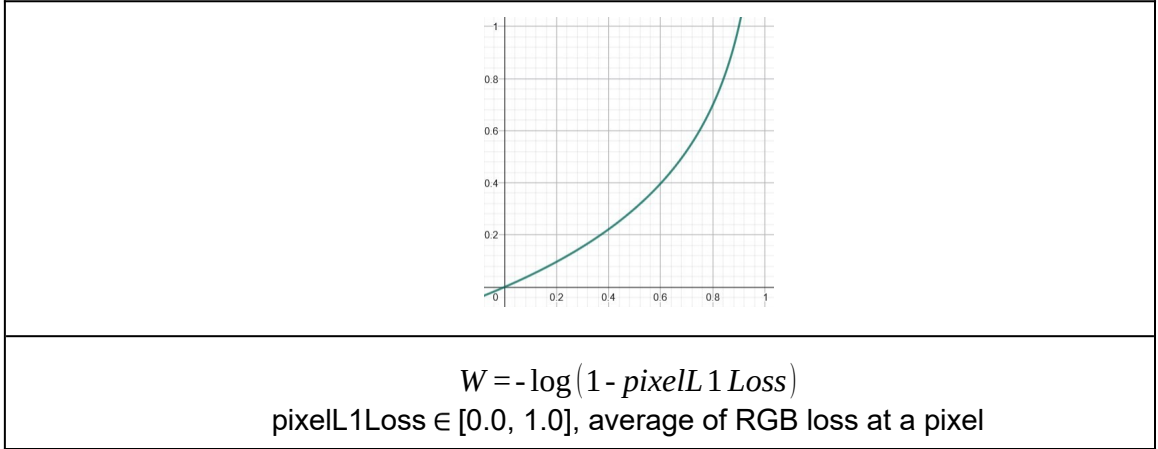


To be complete, the noise in the GS-MCMC strategy is scaled by opacity, as shown in the formula above for $\theta(t+1)$. The more transparent the gaussian, the more noise it gets. The default implementation uses a sigmoid function with a sharp cut-off value for opacity. We feel this scaling is appropriate.

2. **Adaptive Pixel-Loss-Based Repositioning** — In addition to the above noise, GS-MCMC will periodically (every 100 epochs) force transparent gaussians to the positions of opaque gaussians. Candidate targets are randomly chosen based on their opacity, with more opaque gaussians being more commonly selected.

GSplat will store the 2D image space coordinates for each gaussian when they are rendered. Because we have these coordinates, we can find the total RGB loss at that gaussian's 2D location. If that loss is high, then that indicates more help is needed in that area.

Our method assigns an initial weight of $-\log(1 - \text{pixelL1Loss})$ to each gaussian, which influences the likelihood it is selected as a reposition target. The goal was to make it more likely that gaussians with high loss are selected in this repositioning step.



3. **Volume Regularization** — GS-MCMC employs a regularization term in its loss function based on average gaussian dimension. This is meant to keep gaussians from becoming too large. The original uses $\lambda_{scale} \cdot \text{avg}(e^s)$, where s is the length of a dimension. We simply changed this to a regularizer based on estimated volume, making the regularizing coefficient a hyperparameter.

$$\text{Regularizer} = \lambda_{scale} \cdot \text{avg}(e^{x+y+z}) = \lambda_{scale} \cdot \text{avg}(e^x e^y e^z)$$

3.1 Evaluation Metrics

Commonly used metrics in Gaussian Splatting include a mix of performance metrics (computation time and memory), and various metrics for judging the similarity between two images. Typical comparison points are at 7000 epochs and 30000 epochs.

Computation Time — A simple, but practical measure that's not too hard to record and is included in every Gaussian Splatting evaluation. Gaussian Splatting makes extensive use of CUDA kernels and tiled rendering techniques to improve the time spent in computation. Since Gaussian Splatting typically involves a fixed number of epochs to iterate over, we do not expect this to improve, though ideally it should not drastically increase.

Computation Memory — The memory usage of the algorithm, which can also be tracked throughout execution. Smaller memory footprints are better. Again, we do not expect our method to improve on this, but this should tell us if our modifications do not perform well.

Peak Signal to Noise Ratio — One of the simplest metrics to track accuracy for any image processing algorithm, PSNR is calculated based on the mean squared differences between two images. In our case, this is the image produced by Gaussian Splatting and the ground truth.

$$PSNR = -10 \cdot \log_{10}(MSE(trained, test))$$

Structural Similarity [10] — This is used in the traditional Gaussian Splatting loss function, but can also be a metric comparing the end result to ground truth. Structural similarity largely focuses on differences in brightness, contrast, and overall structure, and is meant to be closer to human perception than PSNR. Values range from 0 (not similar) to 1 (structurally equivalent). PyTorch has a built-in object for this which we will use.

Learned Perception Image Patch Similarity [13] — This is a relatively recent image similarity metric, which is used by both [7] and [8]. Like structural similarity, it is a metric that is meant to approximate human perception, and uses pre-trained neural networks to produce a distance score. Smaller numbers are better, and PyTorch comes with objects which will calculate these.

4. Results

Because of limitations in execution time, each set of hyperparameters for each of our three modifications were tuned in isolation, in order to reduce the number of combinations we had to try. We used K-folds with 4 folds for tuning, and initialized the scene from random points to start with maximum error (as opposed to starting with some known set of SfM points). This allowed us to perform an ablation study to see how each of our three modifications performed on their own. We discuss these below.

Execution Time — During hyperparameter tuning, we discovered that our implementation ran much slower than the original. Most GS evaluation is done at 7000 and 30000 epochs, though tuning parameters for that amount of time was prohibitively slow. We instead tuned our hyperparameters for 5000 epochs, then ran a final evaluation at 7000 epochs.

We only experienced this slowdown when tuning the Levy distribution parameters, so the cause of this was the PRNG. PyTorch does not come with native support for the Levy distribution, so our two options were either SciPy or Pyro, a PyTorch library. Despite being coded using PyTorch functions, Pyro was no faster than SciPy, so we ended up using SciPy.

If our approach is evaluated on accuracy per epoch, however, this allows our method to be compared assuming we are not limited by the speed of our PRNG.

Memory Usage — Individually and combined, none of the three modifications increased memory usage by any significant amount. We perform as well as the original for this metric.

Levy-distributed Langevin Dynamics — We found the Levy distribution produced better results with the right hyperparameters. This comes at a severe cost of execution speed, but all three accuracy metrics dramatically increased for our testing scene.

- Noise location: 0.0
- Noise scale: 0.8
- Percentage point function range: 0.0 - 0.9
 - This effectively removes the last 10% of the distribution
 - Values tend to exponentially increase in this range

The following shows values for each of our four folds along with their average (highlighted), against the original GS-MCMC implementation.

Levy-Distributed Langevin Dynamics

Time (sec)	470.74	471.33	473	472.23	471.825
Memory (MB)	1247	1246	1248	1245	1246.5
PSNR	25.27	25.374	25.226	25.417	25.32175
SSIM	0.7936	0.7968	0.7932	0.7999	0.795875
LPIPS	0.277	0.267	0.273	0.264	0.27025

GS-MCMC Baseline

Time (sec)	91.49
Memory (MB)	1231
PSNR	24.121
SSIM	0.7516
LPIPS	0.341

Adaptive Pixel-Loss-Based Repositioning — We did not experiment with any weight functions aside from $W = -\log_{10}(1 - \text{pixelL1 Loss})$, but found this to moderately increase accuracy metrics. Unexpectedly, our execution times were slightly faster with only this modification. Again, the following shows values for each of the four folds, then highlights the average.

Pixel-Loss-Based Adaptive Control

Time (sec)	85.77	86.05	85.62	85.65	85.7725
Memory (MB)	1252	1248	1253	1254	1251.75
PSNR	24.374	24.634	24.661	24.584	24.56325
SSIM	0.7688	0.7735	0.7777	0.775	0.77375
LPIPS	0.312	0.299	0.296	0.3	0.30175

GS-MCMC Baseline

Time (sec)	91.49
Memory (MB)	1231
PSNR	24.121
SSIM	0.7516
LPIPS	0.341

Volume Regularization — Once we settled on an acceptable regularizing coefficient, we found that accuracy only slightly increased, though execution time was also slightly faster. These results were very consistent.

- Regularizing coefficient: 0.025

As always, the results here show each of our four folds, with the average highlighted.

Gaussian Volume Regularization

Time (sec)	88.75	87.56	87.48	87.89	87.92
Memory (MB)	1268	1262	1261	1262	1263.25
PSNR	24.294	24.458	24.468	24.38	24.4
SSIM	0.7569	0.7607	0.7608	0.7592	0.7594
LPIPS	0.334	0.322	0.323	0.327	0.3265

GS-MCMC Baseline

Time (sec)	91.49
Memory (MB)	1231
PSNR	24.121
SSIM	0.7516
LPIPS	0.341

All Modifications — Combining all our modifications, with the hyperparameters tuned earlier, we found a substantial increase in accuracy, and a substantial cost in execution time. However, epoch for epoch, our implementation outperforms the original GS-MCMC method.

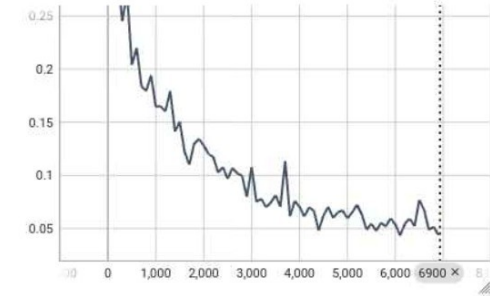
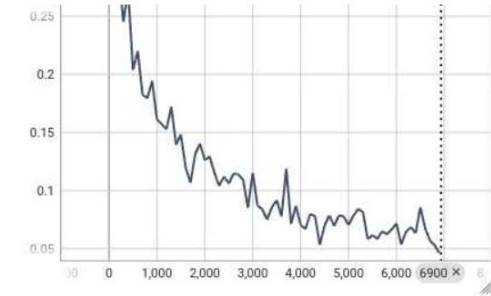
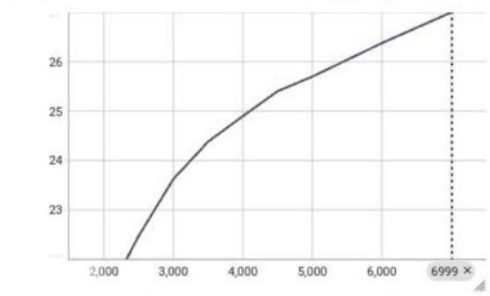
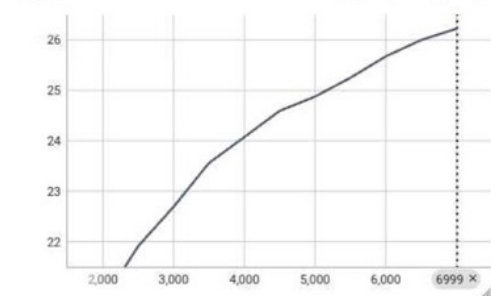
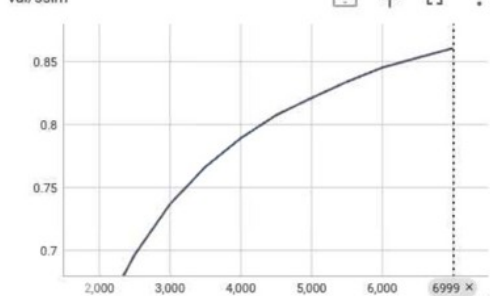
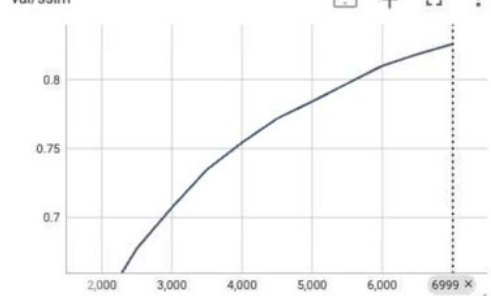
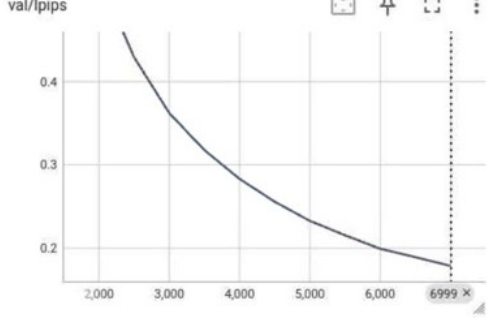
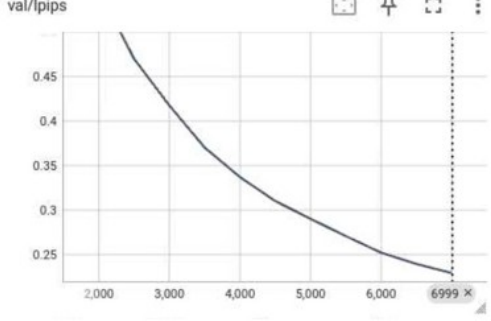
Our Implementation

Time (sec)	593.72
Memory (MB)	1458
PSNR	27.075
SSIM	0.8622
LPIPS	0.176

GS-MCMC Baseline

Time (sec)	108.02
Memory (MB)	1392
PSNR	26.318
SSIM	0.826
LPIPS	0.229

Loss/Accuracy Plots (note that the scale values in the y-axes differ)

Our Implementation	GS-MCMC Baseline
train/loss 	train/loss 
val/psnr 	val/psnr 
val/ssim 	val/ssim 
val/lpips 	val/lpips 

5. Discussion and Future Work

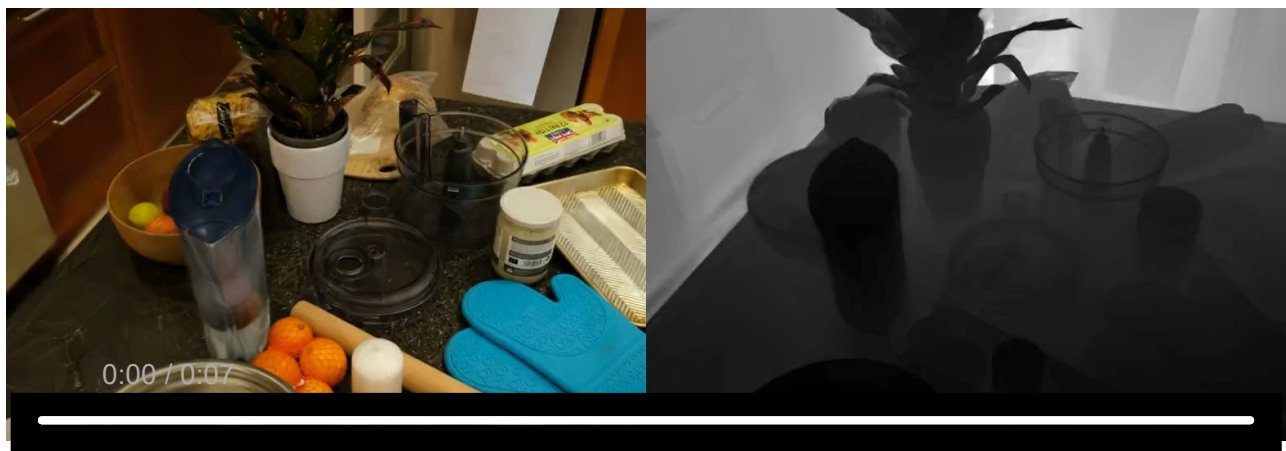
At higher epochs, we actually do expect the two methods to converge to around the same accuracy values as they both approach an acceptable solution. This is a natural part of training in general, given two models that produce good results.

If there are other reasons this could be the case, it's possible our method works best when there's currently a lot of error. Our method prefers to push transparent gaussians away from their current area (which is why we used the Levy distribution), rather than keeping them close (as the normal distribution prefers). It's possible that our more aggressive strategy works best when there's currently a lot of error. As the reconstruction approaches its end and needs only minor adjustments, it's possible the more centered approach of the normal distribution is more suitable.

Also, due to our short development cycle, the hyperparameters were pretty much tuned on a single scene, and for a small number of epochs, then briefly tested on others to see if it produced a generalizable solution. While initial signs show that it does generalize, a full investigation would require more thorough testing and tuning on many scenes and higher epoch values. We've already discussed the limitations we were working with, however.

Execution time was the biggest barrier to our experiments, and in practical terms, one could run the original, much faster GS-MCMC model for more epochs to achieve similar results. However, it may be possible to use a normal distribution and transform it into a distribution that looks similar to the Levy distribution, or at least a distribution that shares the same concept of pushing points out further. This would retain the normal distribution's speed advantage, and combine it with a faster way of testing our hypotheses.

Our metrics were run using the counter scene from the Mip-NeRF 360 dataset. This dataset is rather well-behaved - the scene is static (i.e. there are no changes in the scene, though it's taken from several angles), the images are generally centered in the same spot, and lighting conditions are constant. Our results for this step, at 7000 epochs, can be seen a video in the repository (results-final-7000.mp4).



5.1 Stress Test

As a final test, we ran both the original GS-MCMC algorithm and our implementation through an image set of the Taj Mahal. These images were taken from a public dataset [1], and the camera poses were extracted using a modified COLMAP implementation.

It should be noted this image set is extremely non-ideal. Different people are cluttering up the scene in each shot, some shots are closer or farther, others center on different parts of the Taj Mahal, and they are all taken in different lighting conditions. All these add quite a lot of noise to the dataset. One image was even taken from a tunnel, with much of the picture black except for the Taj Mahal peeking out at the end of the tunnel.

This image set is the sort that Gaussian Splatting will struggle on, and this is exactly what we noticed with the original GS-MCMC algorithm at 7000 epochs. It actually crashed at around 16000, taking an uncharacteristic 13.87 GB of memory.

We then compared this to our implementation, and noticed our accuracy metrics were still higher on average. We only ran 7000 epochs, and expect at higher epochs it will also run out of memory. The original GS-MCMC method had a range of results, at best matching our method, but usually producing worse results. By contrast, our method did not have much variance.

(Averages)	Our implementation	GS-MCMC
Time (sec)	897.38	275.32
Memory (MB)	1751	2268
PSNR	14.239	13.864
SSIM	.6958	.6727
LPIPS	.507	.566

One noticeable difference is that the GS-MCMC results tend to have more gaussians occupying empty space, which occlude the view. Our method seems as though it handles this massive amount of noise better, as there are fewer occlusions in the middle of each frame. There are many frames that neither method can render well, but these frames would be problematic in any Gaussian Splatting algorithm. Unexpectedly, our method uses less memory as well.

All things considered, we feel that our solution is generalizable and produces more accuracy in both ideal scenarios and with noise.

6. References

- [1] “IMC-PT 2020 dataset | Kwang Moo Yi @ UBC,” *www.cs.ubc.ca*.
<https://www.cs.ubc.ca/~kmyi/imw2020/data.html>
- [2] “Mip-NeRF 360: Unbounded Anti-Aliased Neural Radiance Fields,” *Jonbarron.info*, 2022.
<https://jonbarron.info/mipnerf360>
- [3] nerfstudio-project, “GitHub - nerfstudio-project/gspat: CUDA accelerated rasterization of gaussian splatting,” *GitHub*, Jul. 04, 2025. <https://github.com/nerfstudio-project/gspat> (accessed Sep. 30, 2025).
- [4] T. Barron, B. Mildenhall, Dor Verbin, P. P. Srinivasan, and P. Hedman, “Mip-NeRF 360: Unbounded Anti-Aliased Neural Radiance Fields,” *arXiv (Cornell University)*, Nov. 2021, doi: <https://doi.org/10.48550/arxiv.2111.12077>.
- [5] B. Battash and O. Lindenbaum, “Revisiting the Noise Model of Stochastic Gradient Descent,” *arXiv.org*, 2023. <https://arxiv.org/abs/2303.02749> (accessed Nov. 05, 2025).
- [6] N. Brosse, A. Durmus, and E. Moulines, “The promises and pitfalls of Stochastic Gradient Langevin Dynamics,” *arXiv.org*, 2024. <https://arxiv.org/abs/1811.10072> (accessed Oct. 01, 2024).
- [7] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *arXiv.org*, Aug. 08, 2023. <https://arxiv.org/abs/2308.04079>
- [8] S. Kheradmand et al., “3D Gaussian Splatting as Markov Chain Monte Carlo,” *arXiv.org*, 2024. <https://arxiv.org/abs/2404.09591> (accessed Sep. 15, 2025).
- [9] U. Simsekli, L. Sagun, and M. Gurbuzbalaban, “A Tail-Index Analysis of Stochastic Gradient Noise in Deep Neural Networks,” *arXiv.org*, 2019. <https://arxiv.org/abs/1901.06053> (accessed Nov. 05, 2025).
- [10] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, Apr. 2004, doi: <https://doi.org/10.1109/tip.2003.819861> (accessed Nov. 05, 2025).
- [11] M. Welling, D. Bren, and Y. Teh, “Bayesian Learning via Stochastic Gradient Langevin Dynamics.” *Proceedings of the 28th International Conference on International Conference on Machine Learning*, pp. 681–688, Jun. 2021. Available: <https://dl.acm.org/doi/10.5555/3104482.3104568> (accessed Nov. 01, 2025).
- [12] X.-S. Yang, *Nature-Inspired Optimization Algorithms*. Elsevier, 2016. Available: <https://www.sciencedirect.com/book/9780124167438/nature-inspired-optimizationalgorithms> (accessed Nov. 01, 2025).
- [13] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” *arXiv:1801.03924 [cs]*, Apr. 2018, Available: <https://arxiv.org/abs/1801.03924> (accessed Nov. 05, 2025).
- [14] M. Zwicker, H. Pfister, J. van Baar, and M. Gross, “EWA splatting,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 8, no. 3, pp. 223–238, Jul. 2002, doi: <https://doi.org/10.1109/tvcg.2002.1021576>.